

Agent Integrations Are Documents

Driving Systems from Thing Descriptions

Sherif Abdelwahab · Principal Software Engineer, Microsoft

thingctx is open source and my personal work, not a Microsoft product.

github.com/thingctx/thingctx · thingctx.com

Acting agents need self-describing things

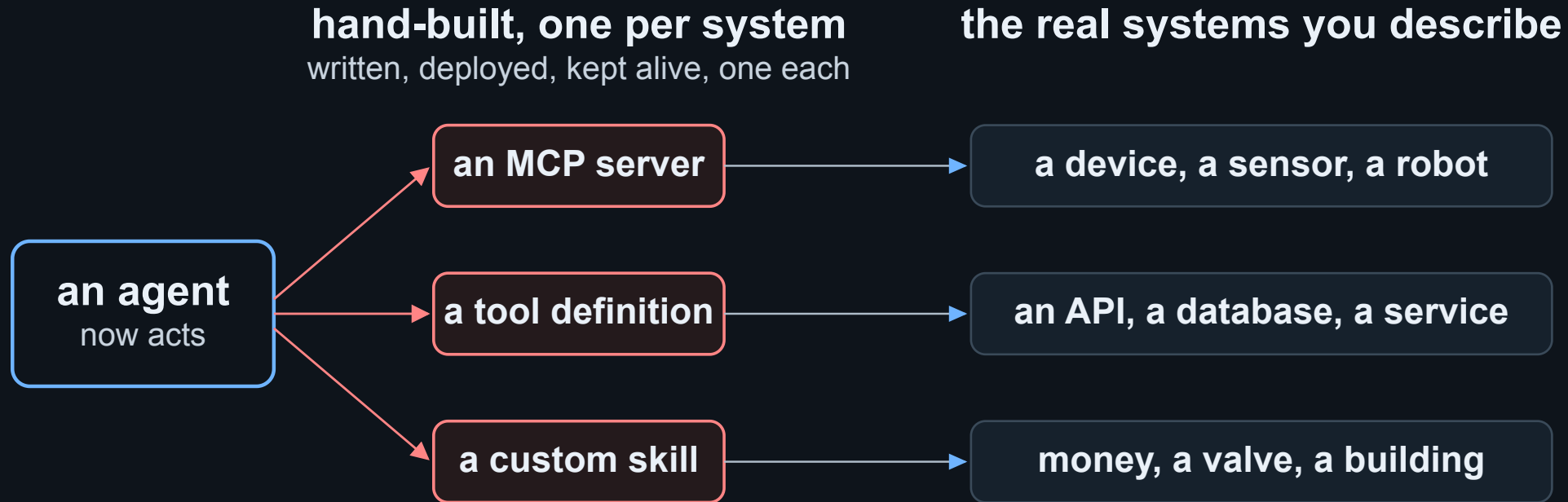
the description lasts longer than the code that reads it:

XDR · WSDL · OpenAPI · protobuf · GraphQL ·

WoT

**a thing describes itself well enough
that any program can use it.**

Today, every integration is hand-built, one per system



the agent no longer answers. it acts on the systems WoT was built to describe.

The missing contract is the TD

A server per system is a missing contract.

no shared way to say what a system does and how to reach it. so everyone rebuilds it.

typing

what is a valid call

security

who, and the secret

transport

how to reach it

events

what it emits



A Thing Description has all four.

you wrote the contract. the agent world hasn't read it yet.

You already wrote the tool

a Thing Description

you already wrote this

```
"createIssue": {  
  "input": {  
    "owner", "repo", "title"  
  },  
  "forms": [{  
    "htv:methodName": "POST"  
  }]  
}
```

the same
object

an agent tool

what the model calls

```
name: github.createIssue  
params: {  
  owner, repo, title  
}  
  
typed. callable. routable.  
nothing added.
```

no new server. no wrapper. the description you have is the tool.

A TD action is an LLM tool

TD action (the document)

```
"createIssue": {  
  "input": {  
    "owner", "repo"  
  },  
  "forms": [...]  
}
```

project

tool spec (model sees)

```
{  
  name: "github.createIssue",  
  parameters: {  
    "owner", "repo"  
  }  
}
```

model calls

resolved HTTP call

```
POST /.../issues  
Authorization:  
Bearer ...  
{ "title": ... }  
bound from the form
```

All three come from the standard. you write none of it

typing

from the input schema

security

from securityDefinitions

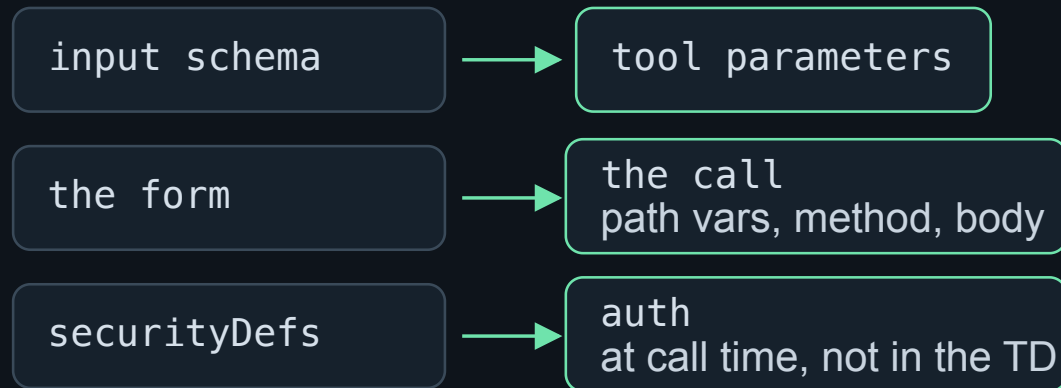
transport

from the form

Resolution is solved; naming is open

Resolution: deterministic

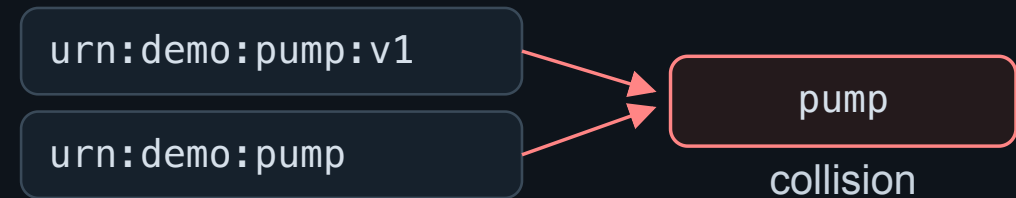
pure functions of the TD. spec-ready.



same TD, same calls, any conforming consumer.

Naming: the open question

not settled. this is what a note must fill.



TD 1.1: affordance names unique only within ONE Thing's map. A fleet shares no namespace.

flatten a fleet to one tool namespace = past the spec.

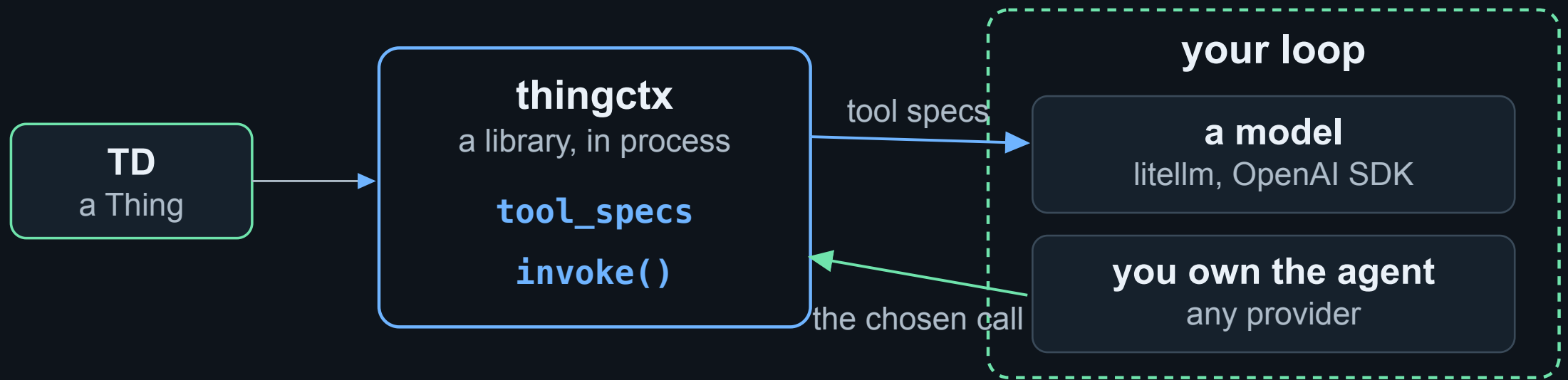
(And the tool channel is a keyhole: how much of a Thing should reach the model is its own open question.)

thingctx drives TDs it has never seen



both runnable in the repo: [examples/interop/](#) (nodewot, ditto)

If you own the agent, the mapping is all you need

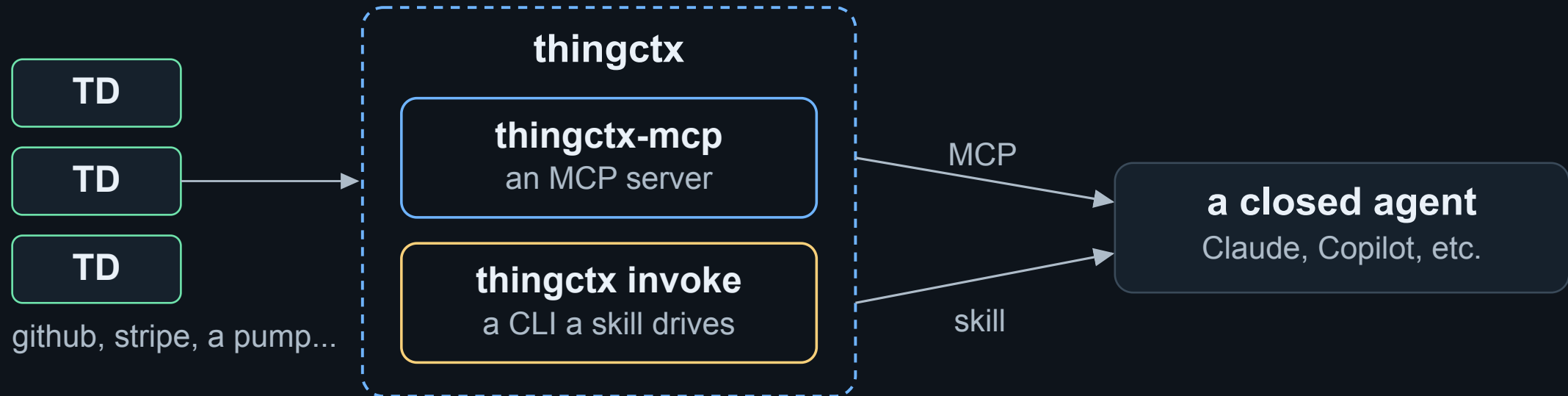


you own the loop, so there is **no server, no bridge**

the mapping alone drives it; thingctx is just a library you call

The same TDs reach a closed agent through any door

a fleet of TDs



the TD feeds both. the door is the vendor's choice

a skill says what to do; thingctx does transport and auth, from the TD

```
thingctx-mcp ./registry/      # one MCP server for the fleet
thingctx invoke ./registry/ github.createIssue --arg ...  # or a CLI a skill drives
```

over MCP, a WoT subscribe becomes resources/updated, buffered against loss.

Only layer one needs the standard

3 · consumer-side handling

event buffering. an MCP limit, not a WoT one.
WoT's event model is already complete.

2 · tc: extensions (optional)

approval, prompts. via WoT's own JSON-LD point.
not a spec change.

1 · the mapping (the part to write down)

read a TD, get a tool.

Needs nothing new from the standard.

Write the TD once; the mapping can change under it

thingctx: the TD stays put

your Thing Description
changes only when the system does.

the mapping evolves underneath it:

map v1 → map v2 → map v3

a new mapping re-authors zero TDs

MCP: change the spec, rewrite

the MCP spec changes
a new protocol version

server 1

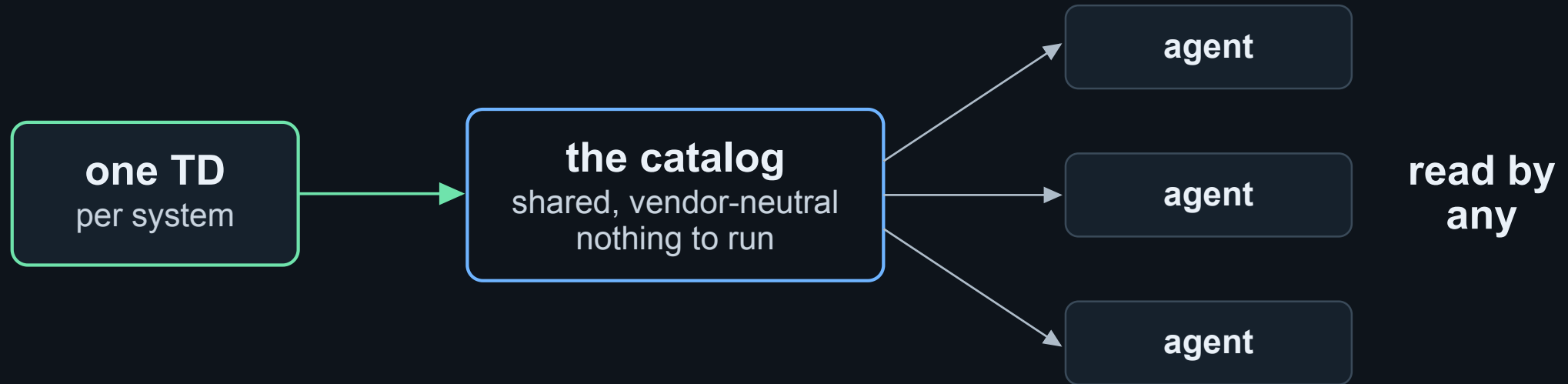
server 2

server N

every author rewrites

the TD stays valid; the server doesn't

Where it is, and what's open



one description per system, in the catalog, reusable by any conforming consumer.

```
pip install thingctx; thingctx-mcp ./registry/
```

open: fleet naming, output schemas, event-over-MCP, who writes the TD

the mapping: docs/MAPPING.md

a binding for a protocol you use: CoAP, OPC UA, Modbus

the catalog: TDs to reuse and add to

Thank you

A Thing Description, as it already exists, is an agent tool.

Sherif Abdelwahab · Principal Software Engineer, Microsoft

find me: github.com/zasherif · linkedin.com/in/zasherif

github.com/thingctx/thingctx · thingctx.com

What "no server" actually costs

per integration	MCP (stdio)	MCP (http)	thingctx
server process	per session	1, long-run	0
hand-written lines	142	142	10
time to first call	540 ms	13 ms	2 ms

one TD, one consumer. ~2 ms to first call because there is no server to start.